UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE ENSINO SUPERIOR DO SERIDÓ
DEPARTAMENTO DE COMPUTAÇÃO E TECNOLOGIA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

Aplicação do Método de Newton para a Solução de Sistemas Não-Lineares

Rusdrael Antony de Araújo Freire

Caicó - RN
2025

RUSDRAEL ANTONY DE ARAÚJO FREIRE

APLICAÇÃO DO MÉTODO DE NEWTON PARA A SOLUÇÃO DE SISTEMAS NÃO-LINEARES

Trabalho de Conclusão de Curso II, apresentado ao Curso de Bacharelado em Sistemas de Informação da Universidade Federal do Rio Grande do Norte, como parte dos requisitos para obtenção do título de Bacharel em Sistemas de Informação.

Orientador(a): Dr. Francisco Márcio Barboza.

Caicó - RN
2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Freire, Rusdrael Antony de Araújo.
Aplicação do Método de Newton para a solução de sistemas não-lineares / Rusdrael Antony de Araújo Freire. - 2025.
42f.: il.

Monografia (Graduação) - Universidade Federal do Rio Grande do Norte, Centro de Ensino Superior do Seridó, Bacharelado em Sistemas de Informação, Caicó/RN, 2025.
Orientação: Dr. Francisco Márcio Barboza.

1. Método de Newton - Monografia. 2. Sistemas Não-Lineares - Monografia. 3. Octave - Monografia. I. Barboza, Francisco Márcio. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira - CRB-15/429

Rusdrael Antony de Araújo Freire

Aplicação do Método de Newton para a Solução de Sistemas Não-Lineares

Trabalho de Conclusão de Curso II, apresentado ao Curso de Bacharelado em Sistemas de Informação da Universidade Federal do Rio Grande do Norte, como parte dos requisitos para obtenção do título de Bacharel em Sistemas de Informação.

Caicó - RN, 13 de janeiro de 2025

Dr. Francisco Márcio Barboza
Orientador
DCT/UFRN

MSc. Jerbeson de Melo Santana
Examinador 1
PETROBRAS

MSc. Arthur Anthony da Cunha Romão e Silva
Examinador 2
FACISA/UFRN

Caicó - RN
2025

Dedico este trabalho aos meus pais e minha namorada, pelo apoio incondicional, e aos meus professores, pelos valiosos ensinamentos. A todos que me acompanharam nesta jornada, minha eterna gratidão.

Agradecimentos

Agradeço primeiramente à minha família, pelo apoio constante e incentivo em todos os momentos. Agradeço também aos meus amigos e colegas, que compartilharam esta jornada comigo. Um agradecimento especial ao meu orientador, Dr. Francisco Márcio Barboza, por sua orientação, paciência e sabedoria, essenciais para a realização deste trabalho. A todos, minha sincera gratidão.

RESUMO

Este trabalho investiga a importância dos sistemas não lineares na ciência e engenharia, destacando a necessidade de métodos eficazes para resolvê-los. O método de Newton, conhecido por sua rápida convergência, é utilizado para aproximar soluções de sistemas de equações não lineares através de iterações baseadas nas derivadas das funções envolvidas, aplicando-se também à otimização para encontrar pontos críticos de funções suaves. O objetivo é explorar o método de Newton para resolver sistemas não lineares $F(\mathbf{X}) = 0$ e apresentar uma implementação prática utilizando o software Octave. Através de exemplos concretos, demonstra-se a eficácia e utilidade do método de Newton em diversos contextos, facilitando a replicação dos resultados e métodos desenvolvidos.

Palavras-chave: Método de Newton, Sistemas Não-Lineares, Octave.

ABSTRACT

This monography investigates the importance of nonlinear systems in science and engineering, highlighting the need for effective methods to solve them. The Newton's method, known for its rapid convergence, is used to approximate solutions of nonlinear equation systems through iterations based on the derivatives of the involved functions, and it is also applied to optimization to find critical points of smooth functions. The objective is to explore Newton's method for solving nonlinear systems $F(\mathbf{X}) = 0$ and to present a practical implementation using Octave software. Through concrete examples, the effectiveness and utility of Newton's method in various contexts are demonstrated, facilitating the replication of the developed results and methods.

Keywords: Newton's Method, Nonlinear Systems, Octave.

LISTA DE FIGURAS

Figura 1 – O método de Newton e as duas primeiras aproximações ao seu zero α. . . . 15
Figura 2 – Falha do método de Newton devido a um ponto inicial ruim. 16
Figura 3 – Gráfico ilustrando o comportamento das iterações no Problema 1. 25
Figura 4 – Gráfico do vetor $\overline{\mathbf{w}}$ sobre o intervalo discretizado. 34

LISTA DE PSEUDOCÓDIGOS

1 Pseudocódigo do Método de Newton em 1D 17
2 Pseudocódigo do Método de Newton para Sistemas Não Lineares 20
3 Pseudocódigo da Jacobiana Númerica 21

SUMÁRIO

1 INTRODUÇÃO 12
1.1 Objetivos 13
1.1.1 Objetivo Geral 13
1.1.2 Objetivos Específicos 13
1.2 Apresentação do Trabalho 13

2 METODOLOGIA 15
2.1 Método de Newton 1D 15
2.1.1 Algoritmo do Método de Newton 1D 16
2.2 Método de Newton Para Sistemas Não-Lineares 18
2.2.1 Algoritmo do Método de Newton para sistemas não-lineares 19
2.2.2 Algoritmo da Jacobiana Numérica 21

3 RESULTADOS E DISCUSSÕES 23
3.1 Problema 1 23
3.2 Problema 2 25
3.3 Problema 3 28

4 CONCLUSÃO 35

REFERÊNCIAS 36

ANEXO A – IMPLEMENTAÇÃO DO MÉTODO DE NEWTON 1D EM OCTAVE 37

ANEXO B – IMPLEMENTAÇÃO DO MÉTODO DE NEWTON PARA SISTEMAS NÃO-LINEARES EM OCTAVE 38

ANEXO C – IMPLEMENTAÇÃO DA JACOBIANA NÚMERICA EM OCTAVE 39

ANEXO D – CÓDIGO DO PRIMEIRO PROBLEMA IMPLEMENTADO EM OCTAVE 40

ANEXO E – CÓDIGO DO SEGUNDO PROBLEMA IMPLEMENTADO EM OCTAVE 41

ANEXO F – CÓDIGO DO TERCEIRO PROBLEMA IMPLEMENTADO EM OCTAVE 42

1 Introdução

O estudo de sistemas não lineares é fundamental em diversas áreas da ciência e da engenharia. Tais sistemas podem descrever uma ampla variedade de fenômenos complexos, desde dinâmicas de populações até cálculo de trajetórias (Burden; Faires, 1997), (Ortega; Rheinboldt, 2000) e (Moré, 1989). A resolução de sistemas de equações não lineares é uma tarefa desafiadora, mas crucial para entender o comportamento desses sistemas e tomar decisões informadas.

Entre os métodos numéricos disponíveis para resolver sistemas não lineares, destaca-se o método de Newton, devido à sua rápida convergência e eficácia em muitos casos. Isso se dá devido à versatilidade do método e à sua capacidade de resolver situações complexas, além de permitir alcançar alta precisão, reduzindo o erro devido à taxa de convergência quadrática do método (Broyden, 1965). Esse método, também conhecido como método de Newton-Raphson, utiliza a derivada das funções envolvidas para iterativamente aproximar a solução desejada. A ideia principal é linearizar o sistema não linear em torno de uma estimativa inicial e resolver o sistema linear resultante, ajustando iterativamente a solução até que a convergência seja atingida (Amaral *et al.*, 2015). A escolha do método de Newton para este estudo se justifica por sua relevância e eficácia na solução de sistemas não lineares, bem como em problemas de otimização (Burden; Faires, 1997; Kelley, 2003). O método é conhecido por sua rápida convergência quando próximo da solução, o que o torna uma ferramenta poderosa para aplicações práticas (Jr; Schnabel, 1996).

Além de sua aplicação na resolução direta de sistemas não lineares, o método de Newton também é amplamente utilizado em problemas de otimização, onde o objetivo é encontrar os mínimos ou máximos de uma função. Em otimização, o método de Newton é adaptado para encontrar os pontos críticos de uma função, onde suas derivadas parciais são iguais a zero (Teodoro, 2013). Esse método é particularmente útil para funções suaves e bem comportadas, onde a matriz Hessiana (a matriz das segundas derivadas) é invertível e fornece informações sobre a curvatura da função, permitindo ajustes mais precisos na direção das iterações (Wright, 2006). Assim, o método de Newton permite uma rápida convergência quadrática, desde que a estimativa inicial esteja suficientemente próxima do ponto crítico e a Hessiana seja definida positiva. No entanto, para problemas de grande escala ou em casos onde a matriz Hessiana não é positiva definida, o método pode ser instável ou computacionalmente caro, sendo necessário recorrer a variantes como o método de Newton modificado ou métodos quase-Newton (Souza, 2015).

Este trabalho tem como objetivo explorar o método de Newton para resolver sistemas não lineares do tipo $F(\mathbf{x}) = 0$, onde $F : \Omega \subset \mathbb{R}^n \to \mathbb{R}^n$ é uma função vetorial. Além disso, será apresentada uma abordagem prática para implementação do método de Newton utilizando o

software Octave.

O uso do software Octave, uma ferramenta de código aberto e amplamente utilizada em ambientes acadêmicos, permite que os resultados e métodos desenvolvidos sejam facilmente replicáveis e acessíveis a outros pesquisadores e profissionais (Octave, 2024). Por meio de exemplos concretos, demonstraremos a aplicação do método em diferentes contextos, evidenciando sua utilidade e eficácia na prática.

1.1 Objetivos

Este tópico delineia os objetivos desta pesquisa, categorizados em objetivo geral e objetivos específicos.

1.1.1 Objetivo Geral

O objetivo geral deste trabalho é explorar e implementar o método de Newton para resolver sistemas não lineares do tipo $F(\mathbf{x}) = 0$, utilizando o software Octave, e demonstrar sua aplicabilidade em problemas reais de diferentes áreas.

1.1.2 Objetivos Específicos

Os objetivos específicos deste trabalho são:

- Revisar a teoria fundamental do método de Newton para sistemas não lineares, incluindo sua formulação matemática e condições de convergência.
- Desenvolver e implementar algoritmos do método de Newton no software Octave.
- Aplicar os algoritmos desenvolvidos a exemplos práticos de sistemas não lineares.

1.2 Apresentação do Trabalho

Este trabalho está estruturado da seguinte forma:

- No Capítulo 1, apresentamos uma revisão teórica sobre o método de Newton, detalhando sua formulação matemática e discutindo as condições sob as quais o método converge.
- No Capítulo 2, descrevemos a metodologia utilizada para a implementação do método de Newton no software Octave, incluindo detalhes sobre a construção dos algoritmos e as estratégias de otimização adotadas.
- No Capítulo 3, apresentamos e discutimos exemplos práticos de aplicação do método de Newton em sistemas não lineares, demonstrando a eficácia e as limitações do método.

- No Capítulo 4, concluímos o trabalho com uma síntese dos resultados obtidos.

2 Metodologia

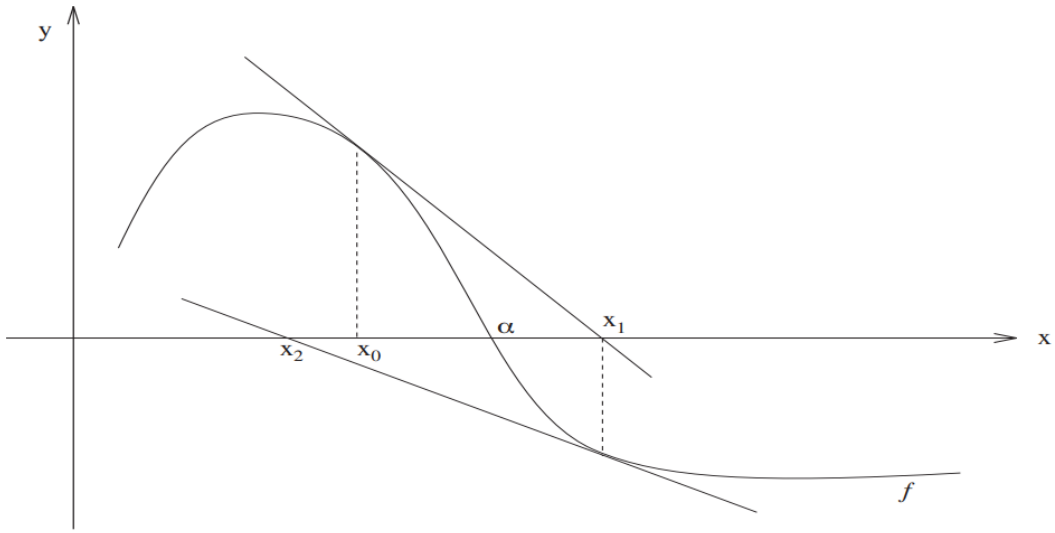
2.1 Método de Newton 1D

Para a resolução de equações, uma das abordagens iterativas mais utilizadas é o método de Newton. Nesse método utiliza-se uma linha tangente à curva. A Figura 1 ilustra uma interpretação gráfica deste método. Para usar o método, começamos com um palpite inicial x_0, suficientemente próximo da raiz α. A próxima aproximação x_1 é dada pelo ponto onde a linha tangente a f em $f(x_0, f(x_0))$ cruza o eixo x. É claro que o valor x_1 está muito mais próximo de α do que o palpite original x_0. Se x_{n+1} denota o valor obtido pelas iterações subsequentes, isto é, a interseção com o eixo x da linha tangente a f em $(x_n, f(x_n))$, então uma fórmula que relaciona x_n e x_{n+1}, conhecida como método de Newton, é dada por

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad n \geq 0 \tag{1}$$

desde que $f'(x_n)$ não seja zero (Kharab; Guenther, 2011).

Figura 1 – O método de Newton e as duas primeiras aproximações ao seu zero α.



Fonte: (Kharab; Guenther, 2011)

Para derivar a Equação 1, observe que a equação da linha tangente em $(x_0, f(x_0))$ é

$$y - f(x_0) = f'(x_0)(x - x_0).$$

Se x_1 denota o ponto onde esta linha intersecta o eixo x, então $y = 0$ em $x = x_1$, ou seja,

$$-f(x_0) = f'(x_0)(x_1 - x_0),$$

e resolvendo para x_1 obtemos

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

Repetindo o processo usando a linha tangente em $(x_1, f(x_1))$, obtemos para x_2

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)}.$$

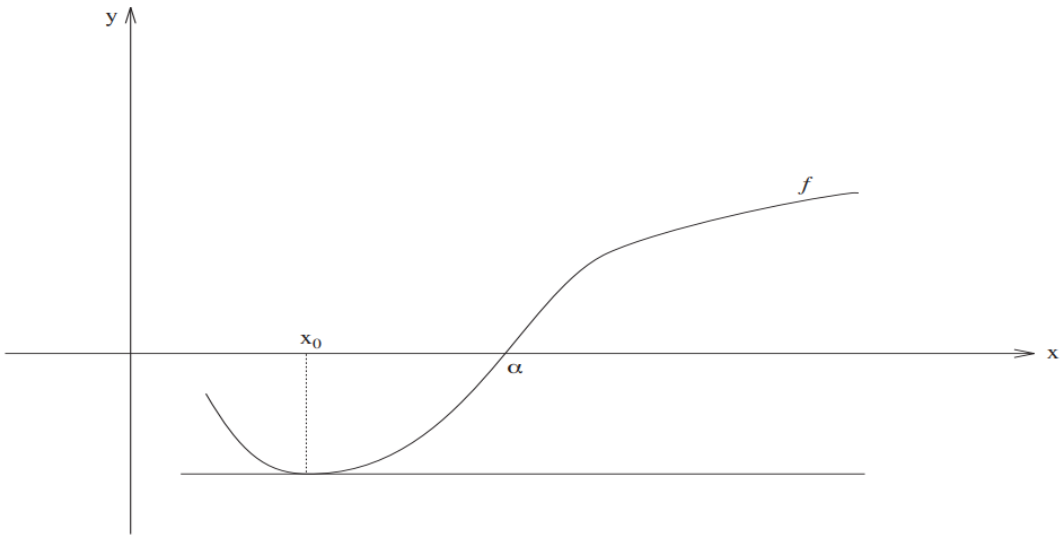
Escrevendo a equação anterior em termos mais gerais, obtém-se a Equação 1. Uma maneira adequada de terminar as iterações é usando um dos critérios de parada sugeridos para o método da secante.

Se definirmos $g(x_n) = x_n - \frac{f(x_n)}{f'(x_n)}$, então a Equação 1 pode ser escrita como

$$x_{n+1} = g(x_n).$$

Assim, segundo Kharab e Guenther (2011), o método de Newton para encontrar a raiz da equação $f(x) = 0$ pode ser visto como a utilização de substituições repetidas para encontrar um ponto fixo da função $g(x)$. O método de Newton é uma técnica poderosa, mas apresenta algumas dificuldades. Uma das dificuldades é que, se x_0 não estiver suficientemente próximo da raiz, o método pode não convergir. É evidente que também não se deve escolher x_0 tal que $f'(x_0)$ esteja próximo de zero, pois, neste caso, a linha tangente é quase horizontal, como ilustrado na Figura 4.

Figura 2 – Falha do método de Newton devido a um ponto inicial ruim.



Fonte: (Kharab; Guenther, 2011)

2.1.1 Algoritmo do Método de Newton 1D

Com base no que foi dito, o processo do Método de Newton em uma variável pode ser descrito mais detalhadamente como segue:

Algorithm 1 Pseudocódigo do Método de Newton em 1D

entrada F**:** A função linear que queremos resolver
entrada DF**:** A função que calcula a derivada de F
entrada x_0**:** O chute inicial para a solução
entrada tol**:** A tolerância para o critério de parada
entrada $maxit$**:** O número máximo de iterações permitidas

```
x ← x_0
iter ← 1
enquanto iter ≤ maxit faça
        f ← F(x)
        df ← DF(x)
        se |df| < ϵ então {Verifica se a derivada é próxima de zero}
                error('Derivada muito próxima de zero. Método de Newton não pode continuar.')
        fim se
        x_n ← x − f/df {Atualiza a solução}
        err ← |x_n − x| {Calcula o erro}
        se err ≤ tol então
                Retorna x_n {Retorna a solução aproximada}
        senão
                x ← x_n
        fim se
        iter ← iter + 1
fim enquanto
Erro: O método de Newton não converge
Retorna x_n {Retorna a última solução calculada}
```

O pseudocódigo apresentado descreve o Método de Newton para resolver uma equação não linear unidimensional. Aqui está uma explicação passo a passo do que cada parte do pseudocódigo faz:

1. **Inicialização:**
   - x é inicializado com o palpite inicial x_0.
   - $iter$ é o contador de iterações, inicializando em 1.
2. **Loop de Iterações:**
   - O loop continua enquanto o número de iterações for menor ou igual a $maxit$.
   - Avalia-se a função F e sua derivada F' no ponto atual x.
   - Verifica-se se o valor absoluto da derivada é menor que um valor muito pequeno ϵ para evitar problemas numéricos.
   - Calcula-se o passo de Newton como o valor da função dividido pela sua derivada e atualiza-se a solução x_n.

- Calcula-se o erro err como o maior valor absoluto das diferenças entre x_n e x.

3. **Verificação de Convergência:**

   - Se err for menor ou igual à tolerância tol, considera-se que a convergência foi alcançada.
   - Se o critério de parada não foi atendido, atualiza-se o valor atual x com a nova solução x_n para a próxima iteração e incrementa-se o contador de iterações.

4. **Retorno da Solução:**

   - Se o número máximo de iterações for atingido sem convergência, retorna-se a última solução calculada x_n.

2.2 Método de Newton Para Sistemas Não-Lineares

O método de Newton, apresentado na seção anterior para localizar os zeros de uma função não linear de uma única variável, pode ser ampliado para funções não lineares em múltiplas variáveis. Contudo, encontrar os zeros de funções não lineares é consideravelmente mais desafiador do que resolver equações lineares. Começando com uma estimativa inicial da solução, o método de Newton consiste em aproximar cada função não linear pelo seu plano tangente; e a raiz comum das equações lineares resultantes fornece a próxima aproximação. Para ver como o algoritmo funciona, primeiro consideramos um sistema de duas equações não lineares. Isto é,

$$f_1(x_1, x_2) = 0$$

$$f_2(x_1, x_2) = 0.$$

Seja $(x_1^{(0)}, x_2^{(0)})$ uma estimativa inicial da solução e f diferenciável em $\mathbf{X}^{(0)} = (x_1^{(0)}, x_2^{(0)})$. A equação do plano tangente à função $y = f_1(x_1, x_2)$ em $(x_1^{(0)}, x_2^{(0)})$ é

$$y - f_1(x_1^{(0)}, x_2^{(0)}) = \frac{\partial}{\partial x_1}[f_1(x_1^{(0)}, x_2^{(0)})](x_1 - x_1^{(0)}) + \frac{\partial}{\partial x_2}[f_1(x_1^{(0)}, x_2^{(0)})](x_2 - x_2^{(0)}).$$

De forma semelhante, a equação do plano tangente à função $y = f_2(x_1, x_2)$ em $(x_1^{(0)}, x_2^{(0)})$ é

$$y - f_2(x_1^{(0)}, x_2^{(0)}) = \frac{\partial}{\partial x_1}[f_2(x_1^{(0)}, x_2^{(0)})](x_1 - x_1^{(0)}) + \frac{\partial}{\partial x_2}[f_2(x_1^{(0)}, x_2^{(0)})](x_2 - x_2^{(0)}).$$

A interseção desses dois planos tangentes com o plano xy é obtida resolvendo o sistema

$$\frac{\partial}{\partial x_1}[f_1(x_1^{(0)}, x_2^{(0)})]\Delta x_1^{(0)} + \frac{\partial}{\partial x_2}[f_1(x_1^{(0)}, x_2^{(0)})]\Delta x_2^{(0)} = -f_1(x_1^{(0)}, x_2^{(0)})$$

$$\frac{\partial}{\partial x_1}[f_2(x_1^{(0)}, x_2^{(0)})]\Delta x_1^{(0)} + \frac{\partial}{\partial x_2}[f_2(x_1^{(0)}, x_2^{(0)})]\Delta x_2^{(0)} = -f_2(x_1^{(0)}, x_2^{(0)}) \tag{2}$$

onde $\Delta x_1^{(0)} = x_1 - x_1^{(0)}$ e $\Delta x_2^{(0)} = x_2 - x_2^{(0)}$.

Resolvemos a Equação 2 por eliminação Gaussiana e a solução resultante $(x_1^{(0)}, x_2^{(0)})$ é usada para obter uma estimativa melhorada da solução. Isto é,

$$x_1^{(1)} = x_1^{(0)} + \Delta x_1^{(0)}$$

$$x_2^{(1)} = x_2^{(0)} + \Delta x_2^{(0)}.$$

Repetimos o processo até que f_1 e f_2 estejam próximos de zero ou $\|\Delta \mathbf{X}\|_\infty = \max_i\{|\Delta x_i|\}$ seja menor que um valor de tolerância especificado, caso em que a convergência foi alcançada. Começando com uma solução aproximada $(x_1^{(i)}, x_2^{(i)})$, a notação vetor-matriz dessas equações é

$$\begin{bmatrix} \frac{\partial f_1(x_1^{(i)},x_2^{(i)})}{\partial x_1} & \frac{\partial f_1(x_1^{(i)},x_2^{(i)})}{\partial x_2} \\ \frac{\partial f_2(x_1^{(i)},x_2^{(i)})}{\partial x_1} & \frac{\partial f_2(x_1^{(i)},x_2^{(i)})}{\partial x_2} \end{bmatrix} \begin{bmatrix} \Delta x_1^{(i)} \\ \Delta x_2^{(i)} \end{bmatrix} = - \begin{bmatrix} f_1(x_1^{(i)}, x_2^{(i)}) \\ f_2(x_1^{(i)}, x_2^{(i)}) \end{bmatrix} \tag{3}$$

e

$$\begin{bmatrix} x_1^{(i+1)} \\ x_2^{(i+1)} \end{bmatrix} = \begin{bmatrix} x_1^{(i)} + \Delta x_1^{(i)} \\ x_2^{(i)} + \Delta x_2^{(i)} \end{bmatrix}, \quad i = 0, 1, 2, \ldots$$

Para um sistema de n equações, podemos escrever o método de Newton generalizando a Equação 3. O sistema generalizado é dado por

$$\begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix} \begin{bmatrix} \Delta x_1^{(i)} \\ \vdots \\ \Delta x_n^{(i)} \end{bmatrix} = - \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \tag{4}$$

onde as funções são avaliadas em $\mathbf{X}^{(i)} = (x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)})$. Resolva este sistema para calcular

$$\mathbf{X}^{(i+1)} = \mathbf{X}^{(i)} + \Delta \mathbf{X}^{(i)}, \quad i = 0, 1, 2, \ldots \tag{5}$$

A matriz de coeficientes na Equação 4 é conhecida como a matriz Jacobiana $J(x_1, \ldots, x_n)$. Se a estimativa inicial estiver suficientemente próxima da solução, espera-se que o método de Newton apresente convergência quadrática, desde que a matriz Jacobiana seja não singular (Kharab; Guenther, 2011).

2.2.1 Algoritmo do Método de Newton para sistemas não-lineares

O método de Newton, como discutido anteriormente, envolve resolver iterativamente um sistema de equações não lineares usando aproximações lineares. O processo pode ser descrito pelo seguinte pseudocódigo:

Algorithm 2 Pseudocódigo do Método de Newton para Sistemas Não Lineares

entrada F**:** A função não linear que queremos resolver
entrada JF**:** A função que calcula o Jacobiano de F (pode ser vazia)
entrada $\mathbf{X}_0$**:** O vetor inicial de palpites para a solução
entrada tol**:** A tolerância para o critério de parada
entrada $maxit$**:**O número máximo de iterações permitidas

```
X ← X_0
iter ← 1
h ← 1e^{-6} {Passo pequeno para diferenças finitas}
enquanto iter ≤ maxit faça
    se JF está vazio então
        J ← calcular Jacobiano numericamente usando F, X e h
    senão
        J ← chamar função JF com X
    fim se
    ΔX ← −J^{-1}F(X) {Passo de Newton}
    X_n ← X + ΔX {Atualiza a solução}
    err ← máximo valor absoluto de (X_n − X) {Calcula o erro}
    se err ≤ tol então
        Retorna Xn {Convergência alcançada}
    senão
        X ← X_n
    fim se
    iter ← iter + 1
fim enquanto
Erro: O método de Newton não converge
Retorna Última solução calculada X_n
```

No pseudocódigo acima, a função principal do método de Newton é iterar até que a solução convirja ou o número máximo de iterações seja atingido. Aqui estão os detalhes de cada passo:

1. **Inicialização**:
   - $\mathbf{X}$ é inicializado com o vetor de palpites iniciais $\mathbf{X}_0$.
   - $iter$ é o contador de iterações, inicializado em 1.
   - h é um pequeno valor usado para calcular as diferenças finitas.
2. **Loop de Iterações**:
   - O loop continua enquanto o número de iterações for menor ou igual a $maxit$.
   - Se a função Jacobiana JF não for fornecida, ela é calculada numericamente usando o método descrito na próxima seção. Caso contrário, a função JF é chamada com $\mathbf{X}$ para obter a matriz Jacobiana.

- Resolve-se o sistema linear $-J^{-1}F(\mathbf{X})$ para encontrar o passo de Newton $\Delta\mathbf{X}$.
- A solução é atualizada para $\mathbf{X}_n = \mathbf{X} + \Delta\mathbf{X}$.
- Calcula-se o erro err como o maior valor absoluto das diferenças entre $\mathbf{X}_n$ e $\mathbf{X}$.

3. **Verificação de Convergência**:
   - Se err for menor ou igual à tolerância tol, a função retorna a solução.
   - Se não houver convergência, $\mathbf{X}$ é atualizado para $\mathbf{X}_n$ e o contador de iterações $iter$ é incrementado.
4. **Retorno da Solução**:
   - Se o número máximo de iterações for atingido sem convergência, a última solução $\mathbf{X}_n$ é retornada.

2.2.2 Algoritmo da Jacobiana Numérica

A Jacobiana de uma função vetorial pode ser calculada numericamente usando diferenças finitas. Este método é útil quando a Jacobiana analítica é difícil de obter. O cálculo da Jacobiana numérica pode ser descrito pelo seguinte pseudocódigo:

Algorithm 3 Pseudocódigo da Jacobiana Númerica

entrada f**:** A função vetorial para a qual queremos calcular a Jacobiana
entrada $\mathbf{X}$**:** O vetor de variáveis em torno do qual a Jacobiana é calculada.
entrada h**:** Um pequeno passo utilizado para calcular as diferenças finitas

```
n ← comprimento de X
m ← comprimento de f(X)
J ← matriz m × n inicializada com zeros
para i ← 1 até n faça
        X_incr ← X
        X_incr(i) ← X(i) + h
        f_incr ← f(X_incr)
        f_0 ← f(X)
        J(:, i) ← (f_incr − f_0)/h
fim para
return J
```

No pseudocódigo acima, o cálculo da Jacobiana numérica envolve os seguintes passos:

1. **Inicialização**:
   - n é o número de variáveis independentes, ou seja, o comprimento do vetor $\mathbf{X}$.
   - m é o número de funções no vetor $f(\mathbf{X})$.

- J é a matriz Jacobiana inicializada com zeros, de dimensão $m \times n$.

2. **Iteração sobre Variáveis Independentes**:
   - O loop itera sobre cada variável independente i de 1 a n.
   - $\mathbf{X}_{\text{incr}}$ é uma cópia do vetor $\mathbf{X}$.
   - $\mathbf{X}_{\text{incr(i)}}$ é incrementado pelo passo h.
3. **Cálculo das Diferenças Finitas**:
   - f_{incr} é calculado no ponto $\mathbf{X}_{\text{incr}}$.
   - f_0 é calculado no ponto original $\mathbf{X}$.
   - A derivada parcial para a variável i é calculada como $(f_{\text{incr}} - f_0)/h$ e armazenada na coluna i da matriz J.
4. **Retorno da Jacobiana**:
   - A matriz Jacobiana J é retornada após o loop.

3 Resultados e discussões

Neste capítulo, serão apresentados e discutidos os resultados obtidos após a aplicação do método proposto. Para avaliar o desempenho do algoritmo, utilizaram-se diversos problemas de referência.

3.1 Problema 1

O primeiro problema consiste em um exemplo extraído de Kharab e Guenther (2011). A tarefa é resolver o sistema não linear:

$$f_1(x_1, x_2) = x_1^3 + 3x_2^2 - 21 = 0$$
$$f_2(x_1, x_2) = x_1^2 + 2x_2 + 2 = 0$$

utilizando o método de Newton, com a estimativa inicial $\mathbf{X}^{(0)} = (1, -1)$. As iterações foram realizadas até que

$$\|\Delta\mathbf{X}\|_\infty = \max_i\{|\Delta x_i|\} < 10^{-6}.$$

A matriz Jacobiana do sistema é dada por:

$$J(x_1, x_2) = \begin{bmatrix} 3x_1^2 & 6x_2 \\ 2x_1 & 2 \end{bmatrix}.$$

No ponto inicial $(1, -1)$, o vetor de funções e a matriz Jacobiana tornam-se:

$$F(1, -1) = \begin{bmatrix} -17 \\ 1 \end{bmatrix}, \quad J(1, -1) = \begin{bmatrix} 3 & -6 \\ 2 & 2 \end{bmatrix}.$$

Os diferenciais $\Delta x_1^{(0)}$ e $\Delta x_2^{(0)}$ são obtidos pela solução do sistema:

$$\begin{bmatrix} 3 & -6 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} \Delta x_1^{(0)} \\ \Delta x_2^{(0)} \end{bmatrix} = - \begin{bmatrix} -17 \\ 1 \end{bmatrix}.$$

A solução para os diferenciais é:

$$\Delta\mathbf{X}^{(0)} = \begin{bmatrix} \Delta x_1^{(0)} \\ \Delta x_2^{(0)} \end{bmatrix} = \begin{bmatrix} 1.555556 \\ -2.055560 \end{bmatrix}.$$

Assim, o próximo ponto de iteração é calculado como:

$$\mathbf{X}^{(1)} = \begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1.555556 \\ -2.055556 \end{bmatrix} = \begin{bmatrix} 2.555556 \\ -3.055556 \end{bmatrix}.$$

As iterações seguintes resultaram nos pontos:

$$\mathbf{X}^{(2)} = \begin{bmatrix} 1.865049 \\ -2.500801 \end{bmatrix}, \quad \mathbf{X}^{(3)} = \begin{bmatrix} 1.661337 \\ -2.359271 \end{bmatrix}, \quad \mathbf{X}^{(4)} = \begin{bmatrix} 1.643173 \\ -2.349844 \end{bmatrix}.$$

Os resultados das iterações estão resumidos na Tabela 1:

Iteração i	$\mathbf{X}^{(i)}$	$\|\Delta\mathbf{X}\|_\infty$
0	[1, -1]	
1	[2.555556, -3.055556]	2.055556
2	[1.865049, -2.500801]	0.690507
3	[1.661337, -2.359271]	0.203712
4	[1.643173, -2.349844]	0.018164
5	[1.643038, -2.349787]	0.000135
6	[1.643038, -2.349787]	7.3×10^{-9}

Tabela 1 – Iterações do Método de Newton para o Problema 3.1.

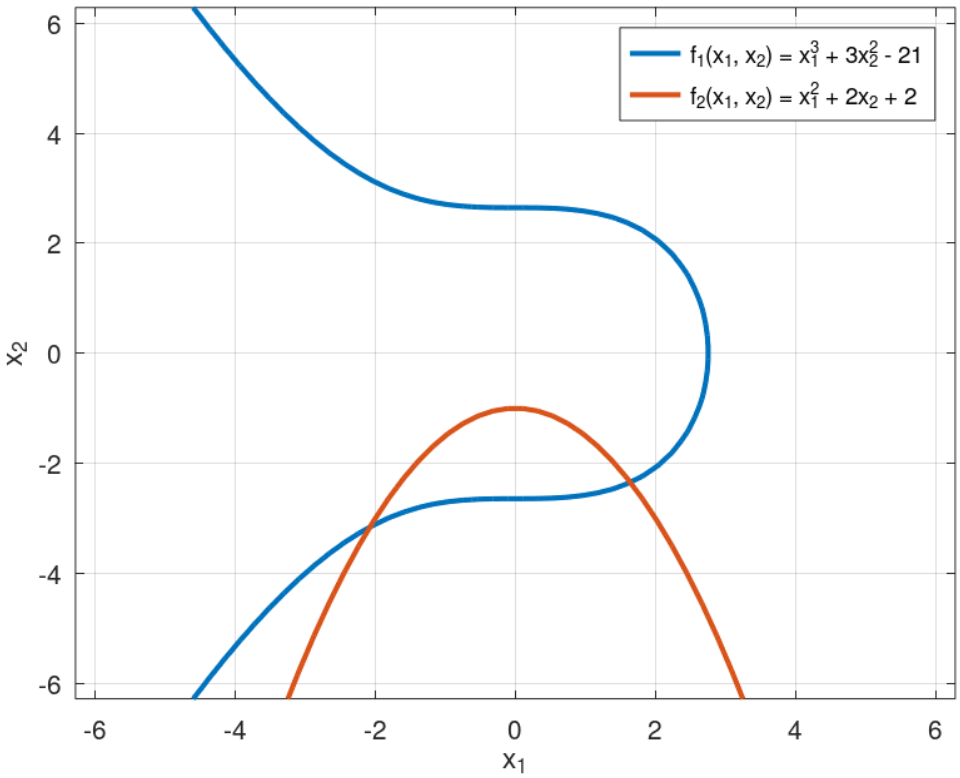


Figura 3 – Gráfico ilustrando o comportamento das iterações no Problema 1.

Fonte: Elaborado pelo autor (2025)

3.2 Problema 2

Este problema é baseado em Remani (2013), com o objetivo de resolver o sistema:

$$
\begin{aligned}
3x_1 - \cos(x_2 x_3) - \frac{1}{2} &= 0, \\
x_1^2 - 81(x_2 + 0.1)^2 + \sin x_3 + 1.06 &= 0, \\
e^{-x_1 x_2} + 20x_3 + \frac{10\pi - 3}{3} &= 0,
\end{aligned}
$$

com a estimativa inicial

$$
\mathbf{X}^{(0)} = \begin{bmatrix} 0.1 \\ 0.1 \\ -0.1 \end{bmatrix}
$$

a estimativa inicial foi encontrado por tentativa e erro à partir de uma análise do sistema de equações

Passo 1: O vetor inicial é

$$\mathbf{X}^{(0)} = \begin{bmatrix} 0.1 \\ 0.1 \\ -0.1 \end{bmatrix}$$

Passo 2: Definimos $F(\mathbf{X})$ e $J(\mathbf{X})$:

$$F(\mathbf{X}) = \begin{bmatrix} 3x_1 - \cos(x_2x_3) - \frac{1}{2} \\ x_1^2 - 81(x_2+0.1)^2 + \sin x_3 + 1.06 \\ e^{-x_1x_2} + 20x_3 + \frac{10\pi-3}{3} \end{bmatrix}$$

$$J(\mathbf{X}) = \begin{bmatrix} 3 & x_3\sin(x_2x_3) & x_2\sin(x_2x_3) \\ 2x_1 & -162(x_2+0.1) & \cos x_3 \\ -x_2e^{-x_1x_2} & -x_1e^{-x_1x_2} & 20 \end{bmatrix}$$

Agora que definimos $F(\mathbf{X})$ e $J(\mathbf{X})$, queremos calcular $F(\mathbf{X}^{(0)})$ e $J(\mathbf{X}^{(0)})$, onde $\mathbf{X}^{(0)} = (0.1, 0.1, -0.1)^\top$:

$$\begin{aligned} F(\mathbf{X}^{(0)}) &= \begin{bmatrix} 0.3 - \cos(-0.01) - \frac{1}{2} \\ 0.01 - 3.24 + \sin(-0.1) + 1.06 \\ e^{(-0.01)} - 2 + \frac{10\pi-3}{3} \end{bmatrix} \\ &= \begin{bmatrix} -1.19995 \\ -2.269833417 \\ 8.462025346 \end{bmatrix} \end{aligned}$$

e

$$\begin{aligned} J(\mathbf{X}^{(0)}) &= \begin{bmatrix} 3 & (-0.1)\sin(-0.01) & 0.1\sin(-0.01) \\ 0.2 & -32.4 & \cos(-0.1) \\ -0.1e^{-0.01} & -0.1e^{-0.01} & 20 \end{bmatrix} \\ &= \begin{bmatrix} 3 & 0.000999983 & -0.000999983 \\ 0.2 & -32.4 & 0.995004165 \\ -0.099004984 & -0.099004983 & 20 \end{bmatrix} \end{aligned}$$

Passo 3: Resolvemos o sistema $J(\mathbf{X}^{(0)})\mathbf{Y}^{(0)} = -F(\mathbf{X}^{(0)})$ usando eliminação de Gauss:

$$\begin{bmatrix} 3 & 0.000999983 & -0.000999983 \\ 0.2 & -32.4 & 0.995004165 \\ -0.099004984 & -0.099004983 & 20 \end{bmatrix} \begin{bmatrix} y_1^{(0)} \\ y_2^{(0)} \\ y_3^{(0)} \end{bmatrix} = - \begin{bmatrix} -1.19995 \\ -2.269833417 \\ 8.462025346 \end{bmatrix}$$

A solução é

$$\mathbf{Y}^{(0)} = \begin{bmatrix} 0.40003702 \\ -0.08053314 \\ -0.42152047 \end{bmatrix}$$

Passo 4: Com os valores de $\mathbf{Y}^{(0)}$, obtemos o próximo ponto de iteração $\mathbf{X}^{(1)} = \mathbf{X}^{(0)} + \mathbf{Y}^{(0)}$:

$$\begin{aligned} \mathbf{X}^{(1)} &= \begin{bmatrix} 0.1 \\ 0.1 \\ -0.1 \end{bmatrix} + \begin{bmatrix} 0.40003702 \\ -0.08053314 \\ -0.42152047 \end{bmatrix} \\ &= \begin{bmatrix} 0.50003702 \\ 0.01946686 \\ -0.52152047 \end{bmatrix} \end{aligned}$$

Podemos usar os resultados de $\mathbf{X}^{(1)}$ para encontrar nossa próxima iteração $\mathbf{X}^{(2)}$ usando o mesmo procedimento.

Passo 5: Repetindo o processo, obtemos as próximas iterações, conforme resumido na Tabela 2:

Iteração i	$\mathbf{X}^{(i)}$	$\|\Delta\mathbf{X}\|_\infty$
0	[0.100000, 0.100000, -0.100000]	
1	[0.499870, 0.019467, -0.521520]	0.421520
2	[0.500014, 0.001589, -0.523557]	0.017878
3	[0.500000, 0.000012, -0.523598]	0.001576
4	[0.500000, 0.000000, -0.523599]	0.000012
5	[0.500000, 0.000000, -0.523599]	8.3×10^{-10}

Tabela 2 – Iterações do Método de Newton para o Problema 3.2.

Conforme mostrado na tabela, a norma $\|\Delta\mathbf{X}\|_\infty$ atinge um valor muito próximo de zero na quinta iteração, indicando que o sistema $F(\mathbf{X}) = 0$ convergiu para uma solução aproximada. Assim, podemos considerar

$$\bar{\mathbf{X}} = \begin{bmatrix} 0.50000000 \\ 0.00000000 \\ -0.52359877 \end{bmatrix}$$

como uma solução aproximada de $F(\mathbf{X}) = 0$.

3.3 Problema 3

Este problema é baseado no trabalho de Remani (2013) e apresenta um problema de valor de contorno envolvendo uma equação diferencial ordinária não linear.

Consideremos o seguinte problema de valor de contorno:

$$y'' = \frac{1}{8}(32 + 2x^3 - yy'), \quad 1 \leq x \leq 3$$

com as condições de contorno:

$$y(1) = 17, \quad y(3) = 14.333333$$

Usamos o passo $h = 0.1$.

Passo 1: Discretização do Intervalo

Dividimos o intervalo $[1, 3]$ em $N + 1 = 20$ subintervalos de largura igual, onde cada valor x_i é calculado como:

$$x_i = a + i \times h, \quad i = 0, 1, 2, \ldots, 20$$

Os valores de x_i para cada i são mostrados na Tabela 3.

i	x_i
0	1.0
1	1.1
2	1.2
3	1.3
4	1.4
5	1.5
6	1.6
7	1.7
8	1.8
9	1.9
10	2.0
11	2.1
12	2.2
13	2.3
14	2.4
15	2.5
16	2.6
17	2.7
18	2.8
19	2.9
20	3.0

Tabela 3 – Valores de x_i para o intervalo $[1, 3]$ com $h = 0.1$

Passo 2: Definição das Condições de Contorno

As condições de contorno são dadas por:

$$w_0 = 17 \quad \text{e} \quad w_{20} = 14.333333$$

Passo 3: Aproximação Inicial

Consideramos uma aproximação inicial $w^{(0)}$ como:

$$w^{(0)} = \begin{pmatrix} 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \\ 15.6666 \end{pmatrix}^t$$

A aproximação inicial foi dada pela a média das condições de contorno $w_0 = 17$ e $w_{20} = 14.333333$

Passo 4: Definição do Sistema Não Linear

Formulamos $F(w)$ como um sistema de 19 equações não lineares:

$$
\begin{aligned}
2w_1 - w_2 + 0.01\left(4 + 0.33275 + \frac{w_1(w_2 - 17)}{1.6}\right) - 17 &= 0 \\
-w_1 + 2w_2 - w_3 + 0.01\left(4 + 0.432 + \frac{w_2(w_3 - w_1)}{1.6}\right) &= 0 \\
-w_2 + 2w_3 - w_4 + 0.01\left(4 + 0.5495 + \frac{w_3(w_4 - w_2)}{1.6}\right) &= 0 \\
-w_3 + 2w_4 - w_5 + 0.01\left(4 + 0.686 + \frac{w_4(w_5 - w_3)}{1.6}\right) &= 0 \\
-w_4 + 2w_5 - w_6 + 0.01\left(4 + 0.84375 + \frac{w_5(w_6 - w_4)}{1.6}\right) &= 0 \\
-w_5 + 2w_6 - w_7 + 0.01\left(4 + 1.024 + \frac{w_6(w_7 - w_5)}{1.6}\right) &= 0 \\
-w_6 + 2w_7 - w_8 + 0.01\left(4 + 1.22825 + \frac{w_7(w_8 - w_6)}{1.6}\right) &= 0 \\
-w_7 + 2w_8 - w_9 + 0.01\left(4 + 1.458 + \frac{w_8(w_9 - w_7)}{1.6}\right) &= 0 \\
-w_8 + 2w_9 - w_{10} + 0.01\left(4 + 1.71475 + \frac{w_9(w_{10} - w_8)}{1.6}\right) &= 0 \\
-w_9 + 2w_{10} - w_{11} + 0.01\left(4 + 2 + \frac{w_{10}(w_{11} - w_9)}{1.6}\right) &= 0 \\
-w_{10} + 2w_{11} - w_{12} + 0.01\left(4 + 2.31525 + \frac{w_{11}(w_{12} - w_{10})}{1.6}\right) &= 0 \\
-w_{11} + 2w_{12} - w_{13} + 0.01\left(4 + 2.662 + \frac{w_{12}(w_{13} - w_{11})}{1.6}\right) &= 0 \\
-w_{12} + 2w_{13} - w_{14} + 0.01\left(4 + 3.04175 + \frac{w_{13}(w_{14} - w_{12})}{1.6}\right) &= 0 \\
-w_{13} + 2w_{14} - w_{15} + 0.01\left(4 + 3.456 + \frac{w_{14}(w_{15} - w_{13})}{1.6}\right) &= 0 \\
-w_{14} + 2w_{15} - w_{16} + 0.01\left(4 + 3.90625 + \frac{w_{15}(w_{16} - w_{14})}{1.6}\right) &= 0 \\
-w_{15} + 2w_{16} - w_{17} + 0.01\left(4 + 4.394 + \frac{w_{16}(w_{17} - w_{15})}{1.6}\right) &= 0 \\
-w_{16} + 2w_{17} - w_{18} + 0.01\left(4 + 4.92075 + \frac{w_{17}(w_{18} - w_{16})}{1.6}\right) &= 0 \\
-w_{17} + 2w_{18} - w_{19} + 0.01\left(4 + 5.488 + \frac{w_{18}(w_{19} - w_{17})}{1.6}\right) &= 0 \\
-w_{18} + 2w_{19} + 0.01\left(4 + 6.09725 + \frac{w_{19}(14.333333 - w_{18})}{1.6}\right) - 14.333333 &= 0
\end{aligned}
$$

Passo 5: Cálculo da Jacobiana $J(w)$

A matriz Jacobiana $J(w)$ é dada por:

$$J(w) = \begin{bmatrix} 2+0.01(\frac{w_2-17}{1.6}) & -1+0.05(\frac{1}{8}w_1) & 0 & \cdots & \cdots & \cdots & \cdots & 0 \\ -1-0.05(\frac{1}{8}w_2) & 2+0.01(\frac{w_3-w_1}{1.6}) & -1+0.05(\frac{1}{8}w_2) & 0 & \cdots & \cdots & \cdots & 0 \\ 0 & a_{32} & a_{33} & a_{34} & 0 & \cdots & \cdots & 0 \\ \vdots & 0 & \ddots & \ddots & \ddots & 0 & \cdots & 0 \\ \vdots & \vdots & 0 & \ddots & \ddots & \ddots & 0 & 0 \\ \vdots & \vdots & \vdots & 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \vdots & \vdots & \vdots & 0 & \ddots & \ddots & a_{i-1j} \\ 0 & 0 & 0 & 0 & 0 & 0 & a_{i,j-1} & a_{ij} \end{bmatrix}$$

Resultados

Os valores das iterações $w^{(k)}$ são mostrados na Tabela 4 até a convergência de $||w^{(4)} - w^{(3)}|| = 0$, indicando que a sequência convergiu.

x_i	w_i	$\mathbf{w}^{(0)}$	$\mathbf{w}^{(1)}$	$\mathbf{w}^{(2)}$	$\mathbf{w}^{(3)}$	$\mathbf{w}^{(4)}$
1.0	w_0	17.0000	17.0000	17.0000	17.0000	17.0000
1.1	w_1	15.6666	16.7657	16.7605	16.7605	16.7605
1.2	w_2	15.6666	16.5183	16.5135	16.5134	16.5134
1.3	w_3	15.6666	16.2664	16.2589	16.2589	16.2589
1.4	w_4	15.6666	16.0102	15.9974	15.9974	15.9974
1.5	w_5	15.6666	15.7504	15.7299	15.7298	15.7298
1.6	w_6	15.6666	15.4879	15.4578	15.4577	15.4577
1.7	w_7	15.6666	15.2240	15.1830	15.1829	15.1829
1.8	w_8	15.6666	14.9609	14.9084	14.9083	14.9083
1.9	w_9	15.6666	14.7011	14.6376	14.6375	14.6375
2.0	w_{10}	15.6666	14.4483	14.3751	14.3750	14.3750
2.1	w_{11}	15.6666	14.2070	14.1267	14.1266	14.1266
2.2	w_{12}	15.6666	13.9835	13.8994	13.8993	13.8993
2.3	w_{13}	15.6666	13.7852	13.7019	13.7018	13.7018
2.4	w_{14}	15.6666	13.6220	13.5444	13.5443	13.5443
2.5	w_{15}	15.6666	13.5060	13.4392	13.4391	13.4391
2.6	w_{16}	15.6666	13.4525	13.4010	13.4010	13.4010
2.7	w_{17}	15.6666	13.4804	13.4475	13.4475	13.4475
2.8	w_{18}	15.6666	13.6132	13.5999	13.5999	13.5999
2.9	w_{19}	15.6666	13.8801	13.8843	13.8843	13.8843
3.0	w_{20}	14.3333	14.3333	14.3333	14.3333	14.3333

Tabela 4 – Resultados das iterações para o valor de w_i

Assim, a solução aproximada do problema de valor de contorno é:

$$\overline{\mathbf{w}} = \begin{pmatrix} 17.0000 \\ 16.7605 \\ 16.5134 \\ 16.2589 \\ 15.9974 \\ 15.7298 \\ 15.4577 \\ 15.1829 \\ 14.9083 \\ 14.6375 \\ 14.3750 \\ 14.1266 \\ 13.8993 \\ 13.7018 \\ 13.5443 \\ 13.4391 \\ 13.4010 \\ 13.4475 \\ 13.5999 \\ 13.8843 \\ 14.3333 \end{pmatrix}^t$$

Cada valor de w_i representa a aproximação de $y(x_i)$, onde $x_i = a + i \times h$ e $i = 0, 1, 2, \ldots, N + 1$.

A seguir, apresentamos o gráfico que ilustra a evolução do vetor $\overline{\mathbf{w}}$ ao longo das iterações. Esse gráfico demonstra visualmente como os valores de w_i se estabilizam, refletindo a convergência do método para a solução aproximada.

Figura 4 – Gráfico do vetor $\overline{\mathbf{w}}$ sobre o intervalo discretizado.

Fonte: Elaborado pelo autor (2025)

4 Conclusão

Neste trabalho, analisou-se a aplicabilidade do método de Newton na resolução de sistemas não lineares. Através de três problemas distintos, observou-se que o método foi efetivo em convergir para soluções precisas, desde que algumas condições fossem atendidas.

No **Problema 3.1**, o método apresentou uma convergência rápida, atendendo ao critério de tolerância estabelecido. A análise das iterações revelou que o algoritmo se mostrou robusto, sendo capaz de corrigir as estimativas iniciais para encontrar a solução em poucas iterações.

No **Problema 3.2**, com um sistema não linear mais complexo, o método de Newton exigiu maior refinamento inicial para evitar divergências. Ainda assim, após ajustes iniciais adequados, o método convergiu eficientemente, demonstrando sua capacidade de tratar sistemas com múltiplas variáveis.

Por fim, no **Problema 3.3**, aplicando o método a um problema de valor de contorno, verificou-se que a discretização e a construção do sistema de equações permitiram a convergência do método, embora tenha demandado mais iterações. Isso evidencia que a aplicação do método de Newton em problemas de valor de contorno é viável, desde que a formulação seja cuidadosamente implementada.

Assim, conclui-se que o método de Newton é uma ferramenta poderosa para a resolução de sistemas de equações não lineares, mostrando-se eficiente em diversos cenários. Contudo, é essencial considerar aspectos como a escolha da estimativa inicial e a estrutura do sistema de equações para garantir uma convergência eficiente.

Referências

AMARAL, C.; SOUZA, M.; CATALAN, T. Um estudo do método de newton-raphson. *Revista eletrônica matemática e estatística em foco*, p. 42, 2015.

BROYDEN, C. G. A class of methods for solving nonlinear simultaneous equations. *Mathematics of computation*, JSTOR, v. 19, n. 92, p. 577–593, 1965.

BURDEN, R. L.; FAIRES, J. D. *Numerical analysis, brooks*. [S.l.]: Cole publishing company Pacific Grove, CA:, 1997.

JR, J. E. D.; SCHNABEL, R. B. *Numerical methods for unconstrained optimization and nonlinear equations*. [S.l.]: SIAM, 1996.

KELLEY, C. T. *Solving nonlinear equations with Newton's method*. [S.l.]: SIAM, 2003.

KHARAB, A.; GUENTHER, R. *An introduction to numerical methods: a MATLAB® approach*. [S.l.]: CRC press, 2011.

MORÉ, J. J. *A collection of nonlinear model problems*. [S.l.], 1989.

OCTAVE, G. A high-level interactive language for numerical computations. *Edition 9 for Octave version 9.1.0.*, 2024.

ORTEGA, J. M.; RHEINBOLDT, W. C. *Iterative solution of nonlinear equations in several variables*. [S.l.]: SIAM, 2000.

REMANI, C. Numerical methods for solving systems of nonlinear equations. *Lakehead University Thunder Bay, Ontario, Canada*, v. 77, 2013.

SOUZA, E. A. de. Métodos iterativos para problemas nao lineares. *Recuperado el*, v. 12, 2015.

TEODORO, M. M. *O método de Newton e fractais*. Tese (Doutorado) — Universidade Federal de Sao Joao del-Rei, 2013.

WRIGHT, S. J. *Numerical optimization*. 2006.

ANEXO A – Implementação do Método de Newton 1D em Octave

```
% Instituição: UFRN
% Aluno: Rusdrael Antony de Araújo Freire
% Orientador: Prof. Dr. Francisco Márcio Barboza
% Trabalho de Conclusão de Curso: Aplicação do Método de Newton para a Solução de Sistemas Não-Lineares

function newton_1d(F, DF, x0, tol, maxit)

  % Validação dos inputs
  if tol <= 0
    error('A tolerância deve ser positiva');
  end
  if maxit <= 0
    error('O número máximo de iterações deve ser positivo');
  end

  x = x0;
  iter = 1;

  while (iter <= maxit)
    f = feval(F, x);
    df = feval(DF, x);

    if abs(df) < eps
      error('Derivada muito próxima de zero. Método de Newton não pode continuar.');
    end

    xn = x + -f / df; % Atualiza a solução
    err = abs(xn - x); % Calcula o erro

    if (err <= tol)
      fprintf('O método de Newton converge após %d iterações para x = %f\n', iter, xn);
      return;
    else
      x = xn;
    end
    iter = iter + 1;
  end

  disp('O método de Newton não converge');
  fprintf('Última solução calculada: x = %f\n', xn);
end
```

Fonte: Elaborado pelo autor (2025)

ANEXO B – Implementação do Método de Newton para Sistemas Não-Lineares em Octave

```
% Instituição: UFRN
% Aluno: Rusdrael Antony de Araújo Freire
% Orientador: Prof. Dr. Francisco Márcio Barboza
% Trabalho de Conclusão de Curso: Aplicação do Método de Newton para a Solução de Sistemas Não-Lineares

function newton(F, JF, x0, tol, maxit)

  % Validação dos inputs
  if tol <= 0
    error('A tolerância deve ser positiva');
  end
  if maxit <= 0
    error('O número máximo de iterações deve ser positivo');
  end

  x = x0;
  iter = 1;
  h = 1e-6; % Passo pequeno para diferenças finitas

  % Tabela
  fprintf('\nTabela: \n');

  while (iter <= maxit)
    % Se JF está vazio, calcule o Jacobiano numericamente
    if isempty(JF)
      J = jacobiana_numerica(F, x, h);
    else
      J = JF(x);
    end

    %y = -J \ F(x0); % Passo de Newton
    y = -inv(J)*F(x);
    xn = x + y; % Atualiza a solução
    err = max(abs(xn - x)); % Calcula o erro

    % Exibe o progresso
    fprintf('Iteração %d: x = [%s], ||Δx||∞ = %e\n', iter, num2str(xn', '%.4f '), err);

    if (err <= tol)
      fprintf('\n O método de Newton converge após %3.0f iterações para \n', iter);
      disp(xn)
      return;
    else
      x = xn;
    end
    iter = iter + 1;
  end

  disp('O método de Newton não converge');
  disp(xn)
end
```

Fonte: Elaborado pelo autor (2025)

ANEXO C – Implementação da Jacobiana Númerica em Octave

```
% Instituição: UFRN
% Aluno: Rusdrael Antony de Araújo Freire
% Orientador: Prof. Dr. Francisco Márcio Barboza
% Trabalho de Conclusão de Curso: Aplicação do Método de Newton para a Solução de Sistemas Não-Lineares

function J = jacobiana_numerica(f, X, h)

  n = length(X); % Número de variáveis
  m = length(f(X)); % Dimensão da saída da função f
  J = zeros(m, n); % Inicializa a matriz Jacobiana

  for i = 1:n
    X_forward = X;
    X_forward(i) = X(i) + h; % Perturbação positiva

    f_forward = f(X_forward); % Calcula f(X + h)
    f0 = f(X); % Calcula f(X)

    % Calcula a derivada parcial usando diferenças finitas progressivas
    J(:, i) = (f_forward - f0) / h;
    end
end
```

Fonte: Elaborado pelo autor (2025)

ANEXO D – Código do Primeiro Problema Implementado em Octave

```
% Institução: UFRN
% Aluno: Rusdrael Antony de Araújo Freire
% Orientador: Prof. Dr. Francisco Márcio Barboza
% Trabalho de Conclusão de Curso: Aplicação do Método de Newton para a Solução de Sistemas Não-Lineares

clc
clear

% Defina o sistema de equações não lineares
F = @(X) [X(1)^3+3*X(2)^2-21;X(1)^2+2*X(2)+2]

% Defina a Jacobiana analítica (opcional)
JF = []

% Chute inicial
x0 = [1;-1];

% Tolerância
tol = 10^(-8);

% Número máximo de iterações
maxit = 40;

% Resolva usando o método de Newton
newton(F,JF,x0,tol,maxit)
```

Fonte: Elaborado pelo autor (2025)

ANEXO E – Código do Segundo Problema Implementado em Octave

```
% Institução: UFRN
% Aluno: Rusdrael Antony de Araújo Freire
% Orientador: Prof. Dr. Francisco Márcio Barboza
% Trabalho de Conclusão de Curso: Aplicação do Método de Newton para a Solução de Sistemas Não-Lineares

clc
clear

% Defina o sistema de equações não lineares
F = @(X) [3 * X(1) - cos(X(2) * X(3)) - 1/2;
          X(1)^2 - 81 * (X(2) + 0.1)^2 + sin(X(3)) + 1.06;
          exp(-X(1) * X(2)) + 20 * X(3) + (10 * pi - 3) / 3]

% Defina a Jacobiana analítica (opcional)
JF = []

% Chute inicial
x0 = [0.1 ;0.1 ;-0.1];

% Tolerância
tol = 10^(-8);

% Número máximo de iterações
maxit = 40;

% Resolva usando o método de Newton
newton(F,JF,x0,tol,maxit)
```

Fonte: Elaborado pelo autor (2025)

ANEXO F – Código do Terceiro Problema Implementado em Octave

```
% Instituição: UFRN
% Aluno: Rusdrael Antony de Araújo Freire
% Orientador: Prof. Dr. Francisco Márcio Barboza
% Trabalho de Conclusão de Curso: Aplicação do Método de Newton para a Solução de Sistemas Não-Lineares

clc
clear

% Defina o sistema de equações não lineares
F = @(w) [
    2*w(1) - w(2) + 0.01*(4 + 0.33275 + (w(1) * (w(2) - 17) / 1.6)) - 17,
    -w(1) + 2*w(2) - w(3) + 0.01*(4 + 0.432 + (w(2) * (w(3) - w(1)) / 1.6)),
    -w(2) + 2*w(3) - w(4) + 0.01*(4 + 0.54925 + (w(3) * (w(4) - w(2)) / 1.6)),
    -w(3) + 2*w(4) - w(5) + 0.01*(4 + 0.686 + (w(4) * (w(5) - w(3)) / 1.6)),
    -w(4) + 2*w(5) - w(6) + 0.01*(4 + 0.84375 + (w(5) * (w(6) - w(4)) / 1.6)),
    -w(5) + 2*w(6) - w(7) + 0.01*(4 + 1.024 + (w(6) * (w(7) - w(5)) / 1.6)),
    -w(6) + 2*w(7) - w(8) + 0.01*(4 + 1.22825 + (w(7) * (w(8) - w(6)) / 1.6)),
    -w(7) + 2*w(8) - w(9) + 0.01*(4 + 1.458 + (w(8) * (w(9) - w(7)) / 1.6)),
    -w(8) + 2*w(9) - w(10) + 0.01*(4 + 1.71475 + (w(9) * (w(10) - w(8)) / 1.6)),
    -w(9) + 2*w(10) - w(11) + 0.01*(4 + 2 + (w(10) * (w(11) - w(9)) / 1.6)),
    -w(10) + 2*w(11) - w(12) + 0.01*(4 + 2.31525 + (w(11) * (w(12) - w(10)) / 1.6)),
    -w(11) + 2*w(12) - w(13) + 0.01*(4 + 2.662 + (w(12) * (w(13) - w(11)) / 1.6)),
    -w(12) + 2*w(13) - w(14) + 0.01*(4 + 3.04175 + (w(13) * (w(14) - w(12)) / 1.6)),
    -w(13) + 2*w(14) - w(15) + 0.01*(4 + 3.456 + (w(14) * (w(15) - w(13)) / 1.6)),
    -w(14) + 2*w(15) - w(16) + 0.01*(4 + 3.90625 + (w(15) * (w(16) - w(14)) / 1.6)),
    -w(15) + 2*w(16) - w(17) + 0.01*(4 + 4.394 + (w(16) * (w(17) - w(15)) / 1.6)),
    -w(16) + 2*w(17) - w(18) + 0.01*(4 + 4.92075 + (w(17) * (w(18) - w(16)) / 1.6)),
    -w(17) + 2*w(18) - w(19) + 0.01*(4 + 5.488 + (w(18) * (w(19) - w(17)) / 1.6)),
    -w(18) + 2*w(19) + 0.01*(4 + 6.09725 + (w(19) * (14.333333 - w(18)) / 1.6)) - 14.333333
];

% Defina a Jacobiana analítica (opcional)
JF = [];

% Chute inicial conforme a imagem fornecida
x0 = [
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666,
    15.6666
];

% Tolerância
tol = 1e-8;

% Número máximo de iterações
maxit = 40;

% Resolva usando o método de Newton
newton(F, JF, x0, tol, maxit);
```

Fonte: Elaborado pelo autor (2025)